CAN 网关项目 · 面试题库（v2）

> 全部问题都围绕 `can-gateway` 这个真实项目展开，答案里的每个数字都来自实测记录，
> 出处见 `docs/records/`。**答不上出处的问题，不要说。**

一、项目总述

Q1. 用 30 秒介绍你的项目

答：基于 STM32F103CBT6 的双节点 CAN 网关。两块板在 500 kbps 总线上以 21 ms 周期互发数据帧，带 CRC8 校验和 16 位序号丢帧统计；上层实现了 1 KB 分片块传输（START/FRAG/END + 整包 CRC32 + ACK）；
心跳独立走 0x1xx 车道，3 秒收不到判离线。

追问：为什么不直接两个 UART 对接？→ CAN 有差分抗干扰、多主仲裁、硬件错误管理，工业现场默认选它。

Q2. 这个项目最难的地方是什么

答：不是写协议，是**区分"代码问题"和"硬件问题"**。有一次 `lost` 自己在涨，
先查 CRC 全对 → 定位到序号语义错误 → 修完还涨 → 再定位到 FIFO 每轮只取一帧。
两次都以为是硬件，最后都是代码。反过来也吃过亏：有一块收发器是真的坏了（误接 5V 击穿）。

追问：你怎么建立"这次是硬件还是软件"的判断？→ 用对照实验：换模块、换板子、换供电，一次只动一个变量。

Q3. 如果重做一遍，你会改什么

答：三件事。①第一版就把 CRC 和序号做进去，不要先跑裸数据再补；②接收端一律"取空 FIFO"而不是"每轮一帧"；
③测试计划写在代码之前——我是先跑通再回头补指标，返工了两次。

二、C 语言与内存

Q4. `static` 的三种用法，你在哪用了

答：①修饰局部变量（生命周期延长到全程，只初始化一次）；②修饰全局变量/函数（内部链接，只在本文件可见）；
③C++ 里修饰类成员。项目里用的是 ①：`static uint8_t blkTxBuf[1024]`。

追问：不用 static 会怎样？→ 1 KB 数组放栈上，而启动文件默认 `Stack_Size = 0x400`（1 KB），
直接栈溢出，表现为随机 HardFault，还极难复现。

Q5. `volatile` 是干什么的，什么时候必须加

答：告诉编译器"这块内存可能被外部改变，不要优化、每次都去读真实内存"。
必须加的三类：①中断与主循环共享的变量；②硬件寄存器；③多任务共享且不加锁的变量。

追问：加了 volatile 就线程安全吗？→ 不。它只保证"读得到最新值"，不保证读-改-写是原子操作。

Q6. 结构体的大小怎么算

答：按成员对齐，结构体总大小还要补齐到最大成员对齐数的整数倍。

比如 `{uint8_t a; uint32_t b;}` 在 32 位上是 8 字节，不是 5。

追问：怎么省？→ 按类型从大到小排，或用 `__attribute__((packed))`（但访问可能不高效甚至触发对齐异常）。

Q7. 指针和数组的区别

答：数组名在表达式中退化为指向首元素的指针，但 `sizeof(数组)` 是整个大小，`sizeof(指针)` 是 4。

数组不能赋值、不能自增。

追问：函数参数传数组实际传的是什么？→ 指针。所以函数里 `sizeof` 拿不到数组长度，必须额外传长度。

Q8. 内存越界会怎么样

答：三种表现：①静默改掉别的变量（最难查）；②读写到非法地址触发 HardFault；③在 Flash 上访问会取到垃圾值。

追问：怎么查？→ 先看 HardFault 时的 PC/LR 和栈里的返回地址，再缩小范围；能加断言就先加边界断言。

三、STM32 基础

Q9. 上电到 `main()` 之间发生了什么

答：`Reset_Handler` → `SystemInit()`（配置时钟）→ `__main()`（scatterload：把 `.data` 从 Flash 拷到 RAM、

把 `.bss` 清零）→ `main()`。

追问：全局变量初值存在哪？→ 有初值的（`.data`）存 Flash，启动时拷到 RAM；没初值的（`.bss`）只在 RAM，启动时清零。

Q10. 72 MHz 是怎么来的

答：外部 8 MHz 晶振（HSE）→ PLL ×9 → 72 MHz SYSCLK → AHB 不分频 → HCLK 72 MHz。

APB1 最高 36 MHz，所以 APB1 分频 /2；APB2 不分频，72 MHz。

追问：为什么 APB1 不能跑 72？→ 数据手册规定 F103 的 APB1 最大 36 MHz，超频外设行为不可保证。

Q11. CAN 的时钟为什么是 36 MHz

答：CAN1 挂在 APB1 上，APB1 = 36 MHz。位定时全部基于这个时钟。

追问：如果 APB1 不是 36 MHz 会怎样？→ 500 kbps 可能配不出整数的 BRP，只能接受有误差的波特率。

Q12. `HAL_Delay(20)` 为什么不准

答：它基于 `HAL_GetTick()`，而 SysTick 中断是 1 ms 一次，所以实际等待是 20~21 ms，

再加上循环体开销，实测周期 21.0 ms，折算 47.6 帧/s。

追问：要精确 20.00 ms 怎么办？→ 用定时器更新事件触发发送，或用输出比较；软件延时做不到。

Q13. 中断优先级怎么理解

答：NVIC 里每个中断有抢占优先级和子优先级。抢占优先级高的可以打断正在执行的低优先级中断；

抢占优先级相同则不能打断，按子优先级排队。

追问：中断里能不能调 `HAL_Delay`？→ 不能。它依赖 SysTick 中断，优先级低于当前中断时会死等。

Q14. 看门狗有几种，你用了吗

答：独立看门狗 IWDG（独立时钟 LSI，主时钟挂了也能复位）和窗口看门狗 WWDG（要求在一定窗口内喂狗，能抓"喂太快"）。

这个项目没启用——`.ioc` 里没配。**教训**：当时看到板子周期性复位，第一反应就是"是不是没喂狗"，结果查下来是外部供电问题。

追问：什么时候必须加？→ 产品级必须有，尤其是有"无人值守"场景的。

四、CAN 总线（重点，面试必问）

Q15. 500 kbps 的位定时怎么配出来的

答：先算位时间：1 / 500k = 2 μs。CAN 把一个位切成 N 个时间份额（tq），

`BRP × N = 时钟 / 波特率 = 36 MHz / 500 kHz = 72`。

取 tq = 222.2 ns（BRP = 8），N = 9 → 9 × 222.2 ns ≈ 2 μs。

再把 9 个 tq 分成 Sync(1) + BS1(6) + BS2(2)，采样点 = (1 + 6) / 9 = 77.8%。

追问：能不能取 BRP = 4、N = 18？→ 可以，(BRP, N) 的合法组合是 72 的因数对：(9,8)/(8,9)/(6,12)/(4,18)/(3,24)。

关键：BRP 和 N 必须成对满足乘积 = 72，**不能一半取一套、一半取另一套**——这正是我们最初文档里算错的地方

（BRP = 4 取自 18 tq 方案，N = 9 取自 8/9 tq 方案，结果 4 × 9 = 36 ≠ 72，实际跑成 1 Mbps）。

Q16. 为什么采样点要选 75%~87.5%

答：采样点太靠前，信号还没稳定就采样，容易采到跳变沿；太靠后，留给位内重同步的余量不够。

CiA 对 500 kbps 推荐 87.5%，工业上 75%~87.5% 都常见。本工程取 77.8%。

追问：两端采样点必须一致吗？→ 不必。CAN 只要求**位时间一致**，采样点各自独立。

PCAN-View 默认 88%，我们板子是 77.8%，一样通信正常。

Q17. SJW 是干什么的

答：重同步跳转宽度。允许接收方在一次位时间内把采样点前后挪动最多 SJW 个 tq，用来吸收晶振漂移。

本工程 SJW = 1 tq。

追问：SJW 取大了会怎样？→ 抗漂移强了，但对噪声更敏感，可能误判跳变沿。

Q18. CAN 的仲裁是怎么做的

答：线与逻辑，显性位（0）会覆盖隐性位（1）。多个节点同时发送时，谁发隐性位谁就退出，改在下一次重发。

ID 越小优先级越高。整个过程不丢数据、不需要软件干预。

追问：仲裁失败会丢帧吗？→ 不会丢，硬件自动重发（前提是 AutoRetransmission 打开）。

Q19. CAN 的错误计数和三种状态

答：每个节点有发送错误计数 TEC 和接收错误计数 REC。

- 正常（Error Active）：TEC/REC < 128，出错时发主动错误帧
- 被动（Error Passive）：某个计数 > 127，只发被动错误帧，不再干扰总线

- 离线（Bus-Off）：TEC > 255，节点自动脱离总线

追问：怎么恢复？→ 硬件检测到 128 次连续 11 个隐性位后自动恢复（需使能 ABOM），否则要软件重新初始化。

Q20. 终端电阻为什么是 120 Ω，为什么在两端

答：双绞线是传输线，两端不匹配会反射。120 Ω 接近线缆特性阻抗，能把能量吸收掉。

两端各一个，并联后直流电阻 60 Ω。

追问：怎么验证接对了？→ 断电，万用表量 CANH－CANL 应约 60 Ω。我们实测 60.6~60.7 Ω，多出的 0.6 Ω 是表笔线和接触电阻。

Q21. 过滤器怎么配的

答：用掩码模式、32 位、FilterBank 0、挂在 FIFO0、Mask 全 0（接收全部），

`SlaveStartFilterBank = 14`（F103 双 CAN 时，CAN1 用到 14 号，其余留给 CAN2）。

追问：想只收 0x200~0x2FF 怎么写？→ ID = 0x200，Mask = 0x700（只比较高 3 位），后续位不关心。

Q22. CAN 的接收 FIFO 有多深

答：F103 每路接收 FIFO 3 级深。**这是个大坑**：如果每轮主循环只取一帧，而发送端周期和你相当，一旦某一轮被拖长就会溢出丢帧。

追问：我们怎么改的？→ 把 `if (HAL_CAN_GetRxMessage(...) == HAL_OK)` 改成 `while (...)`，一次把 FIFO 取空，取空后返回 `HAL_ERROR` 自然退出。改完 A 板跑 7380 帧、B 板 7185 帧，`lost` 全程为 0。

五、协议设计

Q23. 为什么帧里要有序号

答：CAN 硬件能保证"收到的帧是对的"（CRC + ACK），但**不能保证"该来的帧都来了"**。

序号是软件层唯一的丢帧证据。

追问：怎么算丢了几帧？→ `gap = (uint8_t)(本次 seq - 上次 seq - 1)`。gap = 0 连续，1~127 是真丢了这么多，≥128 判为对方重启。

Q24. 为什么 `(uint8_t)(s - lastSeq - 1)` 能自动处理回绕

答：无符号减法溢出会自动取模 256。255 → 0 时，`0 - 255 - 1 = -256`，按 uint8_t 解释就是 0，正确。

追问：如果换成 int 会怎样？→ 会算出 -256，判断全乱。**类型选错，这段逻辑就是错的。**

Q25. 为什么阈值是 128

答：序号只有 8 位，一半是 128。小于 128 的缺口"更可能是真丢了"，大于 128 的缺口"更可能是对方重启、序号归零"。

本质是在两种解释之间取最大似然。

追问：有什么代价？→ 如果真的一口气丢了 128 帧以上，会被误判成重启（只重新对齐、不计数）。

本工程 47.6 帧/s，128 帧 ≈ 2.7 秒，所以拔线测试别超过 2.5 秒。

Q26. 为什么序号只在发送成功时才加

答：这是第一版最大的 bug。当时写成"组帧 → `seq++` → 发送"，结果发送失败（邮箱满/仲裁失败）时序号照样加了，

对端看到缺口就记丢帧——**丢的是一帧从来没发出去过的数据**，我把它叫「幻影丢帧」。

改成只在 `HAL_CAN_AddTxMessage` 返回 `HAL_OK` 时 `seq++`，`lost` 立刻从持续上涨变成 0。

追问：这说明什么？→ **计数器的语义必须和物理事实对齐**。"第几次尝试发"和"第几帧真发出去了"是两回事。

Q27. 为什么 CRC8 和 CRC32 都要

答：两级保护，覆盖的东西不同。

- CRC8：保护**单帧 7 字节**，防止总线干扰造成单帧数据错误（CAN 硬件 CRC 保护的是整帧传输，但上层应用数据仍可能被改）
- CRC32：保护**整个 1 KB 块**，防止分片丢、乱序、拼接错误

追问：CAN 已经有 CRC 了，为什么还要上层 CRC？→ 硬件 CRC 只保证"这一帧在总线上没被破坏"，不保证"这一帧的内容在应用层是对的"（比如内存被踩、代码 bug 填错数据）。

Q28. CRC 参数为什么这么选

答：CRC8 用多项式 0x07、初值 0、不反射、结果不异或（就是 CRC-8/ATM）。

CRC32 用多项式 0x04C11DB7，反射写法 0xEDB88320，初值和结果都异或 0xFFFFFFFF（标准 CRC-32/ISO-HDLC）。

追问：为什么用逐位算法不用查表？→ 1 KB 只要几毫秒，Flash 省 1 KB 比省那点时间划算。

Q29. 心跳为什么单独占一个 ID

答：一开始想和数据帧共用，结果 PCAN-View 里同一行被两种帧轮流覆盖，**根本看不出谁停了**。

拆成独立 ID 后，数据 0x201、心跳 0x101 各占一行，一眼就能看出"数据在跑但心跳停了"。

追问：这说明什么？→ **总线 ID 不只是地址，也是频道。** 不同语义的帧不要挤在一个 ID 里。

Q30. 离线判定为什么是 3 秒

答：心跳 1 Hz，取 3 个周期。丢 1~2 个心跳不误判，真断了 3 秒内一定能发现。

追问：如果心跳周期改成 100 ms 呢？→ 阈值也要按比例缩，否则恢复太慢；但周期太短会占用带宽。

Q31. 序号缺口法测不到什么

答：测不到"完全断链"。断线时一帧都收不到，没有新序号就没有缺口可算；

重连后间隔超过 128 又被判成"重启"。所以必须靠**心跳超时**补这个洞。

追问：两种机制怎么配合？→ 缺口法回答"丢了多少"，超时法回答"还在不在"。互补，不能替代。

Q32. 1 KB 块怎么分片的

答：256 片 × 4 字节。为什么 4 字节不是 5？因为一帧 8 字节里要放 type、seq（分片序号）、CRC8，剩 5 字节，留 1 字节做扩展，取 4 字节对齐。

追问：为什么不一片 8 字节？→ 那就没有序号和校验的位置了。

Q33. 块传输为什么会失败

答：接收端判断分片序号的写法是 `if (idx == blkRxExpect) 收下; else 丢弃`。

丢 1 片，后面所有片的 idx 都对不上，即使完好无损也会被丢掉，直到 END 才发现片数不够。

实测：A 侧有一块只收到 218/256 片，就是"丢 1 片 + 连锁作废 37 片"。

追问：这暴露了什么？→ 协议只有"检测"没有"重传"，对丢包的容忍度是 0 片。

Q34. 如果要加重传，你会怎么设计

答：END 之后接收方检查：片数是否 256、CRC32 是否一致、缺了哪些片。

用一个 32 字节的位图（256 片 = 256 bit）标出缺失片，回 NACK；发送方只重发缺失的片。

最多重试 3 次，还不行就整块失败、上报。

追问：为什么用位图不用列表？→ 位图定长 32 字节，一帧装不下可以分 5 帧，逻辑简单、不用动态内存。

Q35. 滑动窗口能提升什么

答：停等协议发一帧等一个 ACK，吞吐被 RTT 卡死；窗口 4 就是连发 4 帧再等确认，吞吐能提升约 4 倍。

代价是接收端要能缓存乱序帧，逻辑复杂。

追问：CAN 上 RTT 很小，值得吗？→ 值得，因为瓶颈不是传播延迟，是"每帧都要等握手"的处理开销。

六、调试与排错（面试官最爱问"你踩过什么坑"）

Q36. 你遇到最诡异的一个 bug

答：`lost` 在没有任何干扰的情况下持续增长，但 CRC 全对、`rxErr` 一直是 0。

最后发现是 `seq++` 的位置错了（见 Q26）。**诡异在于它伪装成硬件问题**——数据是对的，只是"少了几帧"。

Q37. 板子烧完第一次能跑，第二次连不上

答：CubeMX 生成的工程里 `SYS → Debug` 默认是 **No Debug**，`HAL_MspInit()` 里有 `__HAL_AFIO_REMAP_SWJ_DISABLE();`——上电 4 μs 后就把 PA13/PA14 从 SWD 改成了普通 IO。

所以空芯片能烧，烧完就永远连不上。

修复：改成 `Serial Wire`，代码里会变成 `__HAL_AFIO_REMAP_SWJ_NOJTAG();`。

救砖方法：按住 BOOT0 → 按 NRST → 松 NRST → 松 BOOT0，进出厂 bootloader 后 SWD 就回来了。

Q38. Keil 显示 0 Error，但改动没生效

答：外部工具改了 `main.c`，Keil 的编辑缓冲还是旧的，增量构建认为"文件没变"直接跳过编译，

把旧 `.axf` 又链接了一遍。判断标准不是 `0 Error`，而是**有没有出现 `compiling xxx.c` 那一行**、

以及 `Build Time Elapsed` 不为 0、`Program Size` 有没有变化。

修复：关掉工程重开 → Rebuild。

Q39. 一块板收不到任何帧，怎么定位

答：先分清"发"和"收"。PCAN-View 能看到它的帧 → 发送没问题。再看接收端计数：

`rxErr = 0` 且 `lost = 0` 说明**一帧都没进来过**（不是收到错的）。

然后做对照：把这块板的收发器模块换到另一块板上，看问题跟不跟着模块走。

我们那次最后发现是模块被 5V 打坏了——SN65HVD230 是 3.3V 器件，接 5V 会击穿，而且外观完全看不出来。

Q40. 怎么区分"软件问题"和"硬件问题"

答：**一次只动一个变量**。典型手法：

①同一份固件换板子 → 问题跟着板子走就是硬件；

②换模块 → 跟着模块走就是模块；

③换供电 → 跟着电源走就是电源；

④软件自证：读内部计数器（我们读 `cnt` 停在 3，立刻判断出"三个发送邮箱全满 = 从来没被 ACK"）。

Q41. 那个 `cnt` 停在 3 是什么原理

答：F103 有 3 个发送邮箱，帧必须被对端 ACK 才会释放。

`if (HAL_CAN_AddTxMessage(...) == HAL_OK) cnt++;` 一直不涨，说明三个邮箱全占着——**帧发出去了但没人应答**。这比量电压快得多，也可靠得多。

Q42. 量电压时表笔该碰哪

答：要碰金属。排针要顶针尖或背面焊点，端子要顶到线芯，杜邦头挡住的地方量不到。

我们那次量 CANH - CANL，实测 60.6 Ω，说明两个 120 Ω 终端都在、线路通、没有多余终端。

七、工程习惯

Q43. 你怎么保证"没丢帧"这个结论可信

答：三个独立证据。

①板子自报：`lost = 0`、`rxErr = 0`、帧率 47.6；

②分析仪旁路抓：PCAN-View 独立统计帧数，与板子自报对上；

③长时间跑：7380 帧 / 155 秒不掉。

追问：板子自己统计自己可信吗？→ 不完全可信，所以必须有一个"旁路真值"——这就是买 USB-CAN 分析仪的意义。

Q44. 你的记录文档写什么

答：每条记录四节：**预期 / 实际 / 差在哪 / 结论**。

"差在哪"最重要，写现象、排查过程、根因、修复、代价。

比如"拔线 2 秒，B 板 lost 从 0 涨到 103，A 板还是 0"——然后解释 A 为什么是 0。

追问：为什么写"代价"？→ 让人知道这个问题花了多少时间，也是给自己看的——同一个坑别再踩第二次。

Q45. git 提交怎么写

答：动词 + 对象 + 结果，中文就行。比如「修复：seq 只在发送成功时递增」「记录：拔线 2 秒实测丢 103 帧」。

一个提交只做一件事，这样 `git log` 本身就是项目进度表。

追问：为什么要推 GitHub？→ 双非本科最大的劣势是没人背书，仓库是唯一能自己控制的证据。

八、反问与简历

Q46. 面试官问"这部分是你独立做的吗"，怎么答

答：诚实分层。可以说"协议设计、调试、验证是我做的，部分底层驱动参考了正点原子的例程"。

绝不能说"全是我写的"——一追问细节就露馅，而且这是诚信问题。

Q47. 简历上这个项目怎么写

答：

> 基于 STM32F103 的双节点 CAN 网关：500 kbps 总线，位定时 BRP=8/采样点 77.8%；
> 自定义应用层协议（帧头+序号+CRC8），实现丢帧统计与 1 KB 分片块传输（整包 CRC32 + ACK）；
> 连续运行 155 秒 / 7380 帧无丢帧，总线负载 1.0%。

每个数字都要能解释，面试官一定会挑一个问。

Q48. 你还有什么想问我们的

答：挑"能体现你在思考团队技术栈"的问题：

- 团队现在主要用哪家的 MCU？有没有在做国产替代？
- 新人进来第一个月一般做什么？
- 这个岗位偏驱动还是偏应用？

不要问：加班多不多、能不能远程（至少第一轮不要）。

附：12 个必须能白板写出来的东西

1. CAN 标准数据帧的字段顺序
2. 500 kbps 的位定时推导（BRP × N = 72）
3. CRC8 的 5 行循环体
4. `gap = (uint8_t)(s - last - 1)` 为什么对
5. 三个发送邮箱 + 三态错误计数
6. 1 KB = 256 片 × 4 字节的来历
7. 块传输的 START/FRAG/END/ACK 时序
8. 停等 vs 滑动窗口的吞吐差别
9. 中断服务函数的三条规矩（短、不阻塞、不进浮点）
10. `static` 局部变量放在哪（.data/.bss，不是栈）
11. 上电到 main 的四步
12. 怎么用万用表验证终端电阻

本文件随项目演进更新。每个答案都对应 `docs/records/` 里的一条实测记录。